

Éxitos: Reducing the Software Tax for Database Logging through a Shortcut I/O Path

Yanpeng Hu, Li Zhu, Yunxin Yang, and Chundong Wang

School of Information Science and Technology

ShanghaiTech University, Shanghai, China

{huyp, zhuli2023, yangyx12023, wangc}@shanghaitech.edu.cn

Abstract—Logging is a useful and critical technique widely used by databases. Database logging relies on file write and fsync, which have to traverse multiple software layers and pay heavy runtime tax to the underlying file system like Ext4 locating target data blocks and journaling modified file metadata. As the preallocation of storage space for a log file makes a stable file structure and avoids metadata changes, we develop Éxitos, which reorganizes offset-to-block mappings per preallocated log file for quick lookup and creates a direct shortcut from database to device, bypassing almost all software layers. For non-preallocated log files, Éxitos exploits Ext4’s move-extent feature to resemble a preallocation-like effect. Evaluation confirms that Éxitos boosts throughput by 2.1× and 1.6× for OceanBase and MySQL, respectively.

Index Terms—Database Logging, Software Tax

I. INTRODUCTION

Many databases rely on logging to guarantee consistency and durability [1, 2]. After writing data to a log file, databases call the costly fsync to enforce persistence. Such persistent writes make logging incur severe performance cost [3].

Researchers have considered minimizing the cost of database logging. At the hardware level, modern solid-state drives (SSDs) are equipped with the power loss protection (PLP). PLP enables data supposed to be flushed into persistent media to be held in the internal cache of SSD [4, 5]. The hardware time cost is hence largely reduced for fsync, but this amplifies the relative overhead, i.e., *software tax*, imposed by software layers. Correspondingly, to lower the software tax, researchers have optimized fsync at the operating system (OS) level [6]–[10]. At the application level, a few databases preallocate log files using the fallocate system call (syscall) [1, 2]. Preallocation offers two benefits. First, by reserving contiguous disk space in advance, preallocation eliminates the cost of runtime block allocations. Second, for a preallocated file, invoking fdatsync, rather than fsync, generally does not trigger file system (e.g., Ext4) journaling for metadata. However, our quantitative study indicates that, even with these joint multi-level optimizations, logging remains as a serious bottleneck, impacting the performance of database. For example, on a low-latency NVMe SSD with the PLP feature and the latest Ext4, disabling all logging actions for MySQL [2] increases its performance by 109.0%. Particularly, the software tax accounts for 60.2% time for a logging write on average.

On the other hand, state-of-the-art solutions to mitigate the software tax charged on file I/Os encounter adoption barriers due to their implementation complexity and practical limitations. Kernel-bypass techniques, such as SPDK [11] and NVMeDirect [12], demand extensive reprogramming. As to approaches involving hardware and/or OS changes, such as Moneta-D [13] and BypassD [14], their dependencies on specialized hardware gadgets and intrusive kernel modifications elevate deployment costs and may introduce potential security vulnerabilities, limiting their practical utility.

In this paper, we aim to exempt the runtime software tax charged on database logging in an *all-in-software* fashion. We are inspired by the observation that a log file, if preallocated, has a stable structure with fixed per-file offset-to-block mappings that can be neatly reorganized and referenced. Moreover, writing data to it does not alter the file’s structural layout. This avoids space allocation, metadata journaling, and access permission changes. For non-preallocated log files, we think about if the underlying file system such as Ext4 has implicit features that can be used to resemble a preallocation-like effect.

We accordingly develop Éxitos. Éxitos builds a shortcut path that bypasses almost all software layers and delivers data directly from database to storage device. To this end, it has the following key aspects.

- For a preallocated log file, Éxitos extracts and caches the file’s offset-to-block mappings in an in-memory structure called the *Metadata Collector* (Maco). Maco enables Éxitos to promptly determine target blocks without querying the file’s indexing structure (e.g., extent tree in Ext4).
- At runtime, Éxitos establishes a new shortcut I/O path by leveraging eBPF [15] and ioctl [16]. Referring to per-file mappings kept in Maco, Éxitos uses eBPF to intercept and redirect persistent writes from database to device driver, and pushes down data via ioctl interfaces to target blocks.
- For non-preallocated log files, Éxitos manages a pool of *donor* files to pre-reserve storage spaces. Upon an allocation demand to extend a log file, it exploits Ext4’s *move-extent* feature [17] to efficiently transfer a range of blocks from a donor file to the log file, which is then handled in Éxitos’s shortcut I/O path like a preallocated log file.

Ext4 primarily uses move-extent for defragmentation [17].

We utilize it to online transfer blocks and make a preallocation-like effect for Éxitos to process persistent writes for non-preallocated log files. We further optimize Éxitos as follows. First, we register preallocated donor files in the Maco structure to accelerate transfers. Second, we leverage `io_uring` [18] to do asynchronous transfers outside of the critical path. Third, We partition the Maco and donor pool to be distributed per CPU core for scalability. Last but not the least, we deliver data to storage by device-specific commands via `ioctl` interfaces and enable compatibility with SATA and NVMe SSDs.

We prototype Éxitos in Linux kernel 6.6.5 without any hardware change. Experiments confirm that Éxitos significantly reduces the software tax imposed on database logging. For example, compared to the stock storage I/O path, Éxitos achieves up to $2.1\times$ throughput in serving an OLTP write-intensive workload with OceanBase [1] using preallocated log files. It is also performant with MySQL whose binary log (binlog) is non-preallocated but gradually growing. Compared to BypassD lowering the software tax by specialized hardware, Éxitos yields much higher performance.

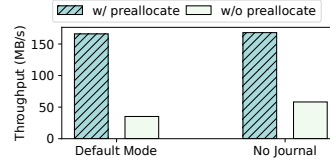
II. BACKGROUND

Databases logging. Many databases rely on logging for consistency and durability. They persistently write new or updated data, or operations, in a log file for recovery in case of power outage or kernel panic [1]–[3]. Log files are hardly read except for recovery, so they can be operated in the direct I/O mode, with the OS’s page cache bypassed. Moreover, some databases, like the rising-star OceanBase [1], initialize log files by preallocating tens of gigabytes by default.

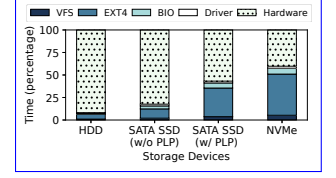
File I/O Path. A file operation traverses a complex stack of multiple software layers before reaching the storage device. The top *syscall interface* transitions syscalls such as file write and `fsync` to the OS kernel with parameter validation. The *virtual file system* (VFS) then performs inode lookup, interacts with the OS page cache, manages file position tracking, and so on. VFS passes the operation to a *specific file system*, e.g., Ext4. Ext4 maps in-file offsets to device block numbers and composes block I/O (bio) requests. For a write targeting unallocated space, Ext4 allocates blocks on demand and updates the per-file mappings. These metadata changes will trigger *journaling* for file system consistency, and Ext4 commits modified metadata to a *journal* before in-place update [8, 9]. The *bio layer* and *I/O scheduler* manage, merge, and schedule bio requests where necessary. The *device driver* translates bio requests to device-specific commands. It finally collects completion or fail signals from the device and notifies upper layers.

Software Tax. Software layers impose heavy tax on the critical path of serving logging writes. Particularly, locating a target block for each write involves a lookup in the file’s extent tree, by which Ext4 stores the offset-to-block mappings per file. The journaling caused by allocating blocks and updating per-file mappings entails severe write amplification with multiple writes performed to the journal and log file [8, 9].

eBPF. The extended Berkeley Packet Filters (eBPF) [15] was initially developed to filter packets in networks. It now



(a) The Impact of Preallocation on File Writes.



(b) The Breakdowns on the File I/O Path.

Fig. 1: The Impact of Preallocation on File Writes and the Breakdowns on the File I/O Path with Varying Devices.

enables advanced capabilities like performance monitoring, security enforcement, and I/O optimization by allowing custom code to run efficiently in the OS kernel. A notable feature of eBPF is its ability to transparently alter the execution path of a kernel function and run alternate codes that have been verified [15, 19]. This makes a seamless transition to a new execution path for a particular syscall, without requiring any change to applications. eBPF also supports a memory buffer shared at the boundary of kernel and user spaces. We will exploit these features to exempt the heavy tax imposed in the software stack on database logging I/Os.

III. MOTIVATION

In this section, we conduct a quantitative study composed of several tests, in order to deeply analyze the software tax that logging writes need to pay. The setup of our testing platform is detailed in Section V.

O1. *The preallocation strategy significantly boosts the performance of database logging writes.*

Database logging depends on persistent file writes. In the first test, we study an SSD (Samsung PM1725a) which has supercapacitor-enabled PLP and features low write latency. We use Fio [20] to create a file and repeatedly append data until it reaches 1GB, with 4KB per request, each followed by `fdatsync`. We configure two file creation options in Fio. One preallocates a 1GB file at start. The other begins with an empty file that grows to 1GB gradually. The file system is Ext4. This test aims to analyze the effect of preallocation.

Figure 1a shows the throughput (bandwidth in MB/s) for writing data to two files when Ext4 is mounted in two modes. In the default `data=ordered` mode, the throughput increases from 35MB/s to 166MB/s. This improvement occurs because preallocation eliminates on-demand block allocation and avoids metadata journaling. Comparatively, for the growing file, each appending write triggers a cascade of operations on the critical path, including the query of block groups for free space, the modification of block bitmap upon allocation, the update of file’s extent tree, and the composition and commit of metadata changes to the journal. These operations construct a serious

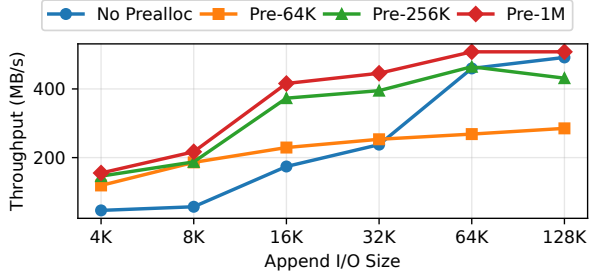


Fig. 2: The Impact of On-the-fly Preallocation on Appending Writes on a Dynamically Growing File.

performance cost that becomes increasingly problematic with fast storage devices like the SSD we are using. Even if we disable journaling, i.e., ‘No Journal’ in Figure 1a, preallocation still yields significant performance improvements. Additionally, preallocation is likely to ask Ext4 to assign contiguous blocks to a file, which is beneficial for the sequential appending I/O pattern of database logging. To sum up, *we shall adopt the preallocation-like strategy to better exploit the device- and OS-level optimizations for accelerating logging writes.*

○2. *The software tax is dominated by file system operations and badly impairs the performance of database logging.*

We next assess the software tax imposed per write with a preallocated file. We use four devices for a comprehensive analysis: a hard disk (WDC WD20EZWX), a SATA SSD without PLP (Samsung 870EVO), a SATA SSD with PLP (Samsung PM893), and an NVMe SSD with PLP (Samsung PM1725a). We measure the time cost of each software layer using Intel VTune Profiler [21]. The I/O pattern is the same as used in the first test with a preallocated file under Ext4 file system. Figure 1b presents the breakdowns for hardware and software layers with four devices. In spite of preallocation, *the runtime software tax increasingly dominates I/O latency as devices become faster, reaching 60.2% for the NVMe SSD.*

Specifically, Ext4 is the predominant latency source with the NVMe SSD, contributing about 46.3%. Although no structural change occurs to the file, frequent metadata operations persist, mainly due to searching for target block numbers in the per-file mappings [14]. *Moving such search operations off the critical path shall effectively reduce the overall software tax.*

○3. *For non-preallocated log files, logging writes still incur severe overhead, which can be mitigated through a preallocation-like strategy.*

Real-world databases may use preallocated or growing log files, such as the binlog of MySQL [2]. As mentioned, an appending write triggers costly runtime operations with a growing file. We investigate if there is a chance that we can accelerate file writes issued to non-preallocated log files in a preallocation-like way. In the third test, we append overall 10GB data to a file in the direct I/O mode with the size per write being 4KB, 8KB, 16KB, 32KB, 64KB, and 128KB. They cover common database transaction sizes for logging in practice. We primarily create an empty file to complete this appending

TABLE I: A Comparison between SOTA Designs.

Solutions	No hardware change	Non-intrusive kernel	Transparent support for applications	Existing file system compatibility	Compatibility with SATA SSD
SPDK	✓	✓	×	×	✓
NVMeDirect	✓	×	×	×	×
Moneta-D	×	×	✓	✓	✓
BypassD	×	×	✓	✓	×
Éxitos	✓	✓	✓	✓	✓

task. The curve with dots in Figure 2 shows a rising throughput with the increase of I/O size. Next, we explicitly apply an on-the-fly preallocation prior to each appending write. That is, we call fallocate to preallocate numerous blocks to the file before appending data. Without loss of generality, we extend the file in 64KB, 256KB, or 1MB at a time. We initialize preallocated blocks by filling zeros after fallocate to truly acquire the space and invoke fsync to trigger journaling in advance. By doing so, we intend to concretely prepare space in a batched manner for one or multiple subsequent appending writes.

In Figure 2, the other three curves with squares, triangles, and diamonds represent the bandwidths with dynamically preallocating 64KB, 256KB, and 1MB, respectively. A comparison among four curves brings about two inspiring points. First, *on-the-fly preallocation, though incurring allocation and journaling overheads on the critical path, is likely to boost write performance.* For example, given the typical I/O size of 4KB, the throughput increases by 2.6×, 3.2×, and 3.4× with three spaces allocated ahead, respectively. Note that preallocating a contiguous space causes file system journaling once, whose cost is much less than the accumulated cost of journaling for every small write. Secondly, *the effect of on-the-fly preallocation diminishes when the I/O size is close to or greater than the preallocation size.* A large I/O itself will trigger space allocation, and preallocating space turns to be neither necessary nor meaningful.

Furthermore, we have thought about what we can do to improve beyond this test. Although *on-the-fly preallocation is effective when combining smaller writes with larger allocated space, Ext4 introduces substantial time cost for each allocation on the critical path* due to space management and metadata journaling. Our design must address this issue (see Section IV).

○4. *State-of-the-art solutions developed to reduce the software tax exhibit considerable limitations.*

Regarding the evolution of storage devices, researchers have studied the software tax charged on file I/Os. Table I shows state-of-the-art (SOTA) solutions in five dimensions. Intel SPDK [11] and NVMeDirect [12] demand extensive reprogramming efforts due to poor POSIX API compatibility, challenging the reliance of databases on standard file I/O interfaces. Moneta-D [13] and BypassD [14] are more compatible with existing file systems and databases. Nevertheless, they require kernel changes and/or specialized hardware or software. Take BypassD for illustration. It handles a request of mapping an in-file offset to target block number through a specialized

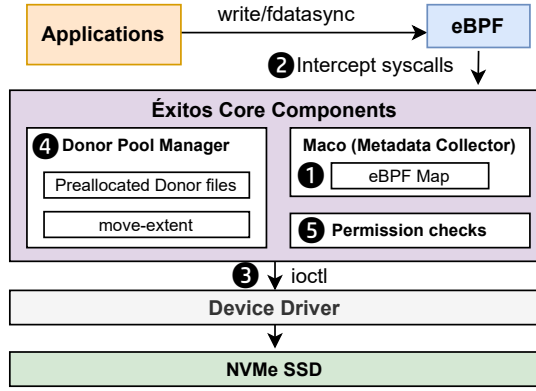


Fig. 3: The Architecture of Éxitos.

IOMMU. Also, its implementation introduces more than 4,200 lines of code (LOC) in the Linux kernel. Furthermore, it only supports NVMe SSD. These hinder its portability and usability. In all, such drawbacks limit the effectiveness of SOTA designs in general-purpose production environments.

IV. THE DESIGN OF ÉXITOS

A. An Overview of Éxitos

Éxitos is specifically designed to build a fast shortcut I/O path for database logging. It is an *all-in-software* solution aiming to reduce the heavy software tax charged on logging writes. Éxitos separates management actions from actual I/O delivery. As depicted by the architecture of Éxitos shown in Figure 3, it intercepts file write and fdatasync for database logging using the eBPF hooks. The intercepted operations are then redirected via ioctl interfaces and delivered to the device driver (Section IV-B).

Éxitos is applicable to both preallocated and growing log files. For preallocated ones, Éxitos extracts per-file offset-to-block mappings at initialization and manages these metadata using an in-memory *Metadata Collector* (Maco) structure. For a non-preallocated log, Éxitos employs a *donor-based preallocation-like strategy* that pre-reserves contiguous block ranges and exploits Ext4’s *move-extent* feature [17] to efficiently extend file space with minimized runtime cost (Section IV-C). This enables Éxitos to handle a growing log file in its fast path.

Furthermore, to make an effectual and self-contained design, we optimize Éxitos in the dimensions of scalability, comprehensiveness, and compatibility (Section IV-D).

The primary technical novelty of Éxitos lies in two points. First, Éxitos builds its shortcut I/O path on per-file mappings that stay stable: once a database log file has been preallocated, the file’s mappings from in-file offsets to device block numbers remain unchanged throughout the period in which the log file stays open. Stable per-file mappings make an all-in-software shortcut both feasible and safe, because the block numbers that Éxitos issues I/Os to are the ones that Ext4 itself has computed. Second, Éxitos turns move-extent from a feature for defragmentation into a means of supplying space online, so that per-file mappings stay equally stable for a growing log file.

B. The Shortcut of Éxitos

Let us take a preallocated log file to illustrate the core functionality of Éxitos. One such file has a stable structure, without any space reallocation or alternation of access permission. Particularly, it has fixed per-file mappings from in-file offsets to device block numbers.

Workflow. Figure 3 decomposes Éxitos’s main components and the workflow that Éxitos follows to handle a persistent write for database logging. It firstly initializes the in-memory Maco structure for the log file (❶ in Figure 3). At runtime, Éxitos intercepts file write and fdatasync syscalls (❷) and proceeds them in its own shortcut path (❸). Next we would detail these three steps.

Metadata management. As mentioned, it is essential but time-consuming for a logging write to locate device blocks by querying the log file’s extent tree. Éxitos duplicates the mappings of a preallocated file in its Maco structure for quick reference. Figure 4 describes an example that a log file receives three contiguous block ranges after preallocation. At initialization, i.e., when opening the log file, Éxitos extracts mappings from the file’s extent tree between in-file offsets (e.g., 4096 and 8192 in Figure 4) and block numbers (resp. 3608 and 5444). Éxitos neatly reorganizes this metadata within an in-memory eBPF map that stores the file descriptor, in-file offsets, base block numbers and lengths for block ranges, thereby enabling prompt block-level addressing. At runtime, regarding block numbers found in the Maco structure, Éxitos redirects write and fdatasync requests to the device driver.

Runtime interception. To intercept file operations, Éxitos employs *syscall tracepoints* that are static kernel instrumentation points characterized by minimal runtime overhead. Tracepoints avoid the cost of dynamic instrumentation, making them ideal for latency-sensitive operations [22]. Éxitos loads and runs these interception programs with bpftrace [23], an eBPF runtime that is compatible with the standard eBPF toolchain. Éxitos obtains a list of log files from a configuration file that the database already reads, and Éxitos intercepts only the write and fdatasync requests issued to the log files on the list. Requests issued to any other file keep taking the conventional path, so a database needs no change to the source code of the database. Before issuing an ioctl request, Éxitos checks that the in-file offset and the length of the request fall inside one of the block ranges that the Maco structure has registered for the file descriptor in question. A request whose in-file offset and length fall outside every registered block range returns to the conventional path for handling instead of entering the shortcut I/O path of Éxitos.

Shortcut path. Database logging relies on file write and fdatasync, and Éxitos particularly reshapes the procedures of these two syscalls. By referring to the Maco structure, Éxitos directly issues data alongside device commands to the driver via ioctl interfaces. It synchronously waits for the return signals of write and fdatasync, to ensure that logging writes orderly complete before the in-place update of database records.

Additionally, the aforementioned PLP feature helps to

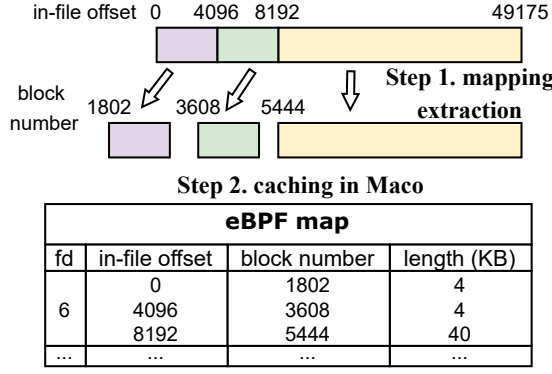


Fig. 4: An Illustration of Éxitos’s Maco.

quickly complete a flush for persistence. If an SSD does not have PLP, Éxitos needs to explicitly invoke a flush command, like the `nvme-flush` of NVMe protocol, to manually drain the SSD’s internal cache for `fdatsync`. This results in the semantic equivalence to original `fdatsync` for guaranteed persistence [24].

C. Donor-Based Online Space Allocation

Using dynamically growing log files is also common for databases. Let us enhance Éxitos to serve databases with such non-preallocated log files. Note that a log file that expands over time demands that Éxitos must track and keep the changing per-file mappings, which is a non-trivial challenge for it. For example, XRP [19], one prior study that attempts to optimize file read, manually updates the cached version of per-file mappings every time some change has happened to a file’s mappings. Note that this is feasible for file read, but infeasible for file write. For a file read, an update of per-file mappings has taken place before the read request, hardly affecting the performance of read operations. Nonetheless, for a file write, the update itself is exactly part of the write procedure and, worse, occurs on the critical path with costly space allocation and file system journaling. As mentioned in Section III, the time cost per write is substantial for a growing file.

To overcome this issue and enable Éxitos to take over non-preallocated log files, we develop a preallocation-like strategy. The core of this strategy is a *donor pool* of large files and the *move-extent* feature of Ext4 [17].

Donor pool. Initially, Éxitos preallocates a pool of donor files. Each donor file is composed of a large contiguous space of blocks, initialized by prefiling zeros. To facilitate an efficient use of blocks in these donor files, Éxitos registers their per-file mappings in the Maco structure in advance.

Moving extents. The *move-extent* is a feature that Ext4 file system uses for online defragmentation [17]. In short, given two files, it atomically exchanges extent mappings in between without copying data, if both files do not have valid data stored in exchanged extents. By doing so, scattered spaces occupied by files can be reclaimed through exchanges and Ext4 achieves the goal of reclaiming fragments. However, we instead exploit this *move-extent* feature to efficiently handle the expansion need

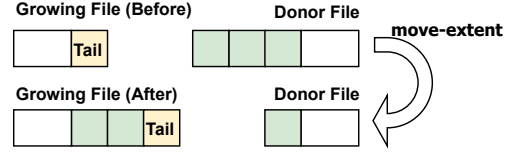


Fig. 5: An Illustration on how Éxitos uses the *move-extent* feature to expand a growing log file.

for a growing file. For one such log file, we intentionally leave an unused block as a special tail, without holding any valid data. When a database intends to append data to unavailable space in the log file and raises a request for space allocation, Éxitos redirects the allocation to call *move-extent* that swaps a range of blocks filled with zeros in a donor file with the tail block of the log file. After the space exchange, the growing log file makes the last received block be the new tail.

Figure 5 briefly illustrates how Éxitos leverages the *move-extent* feature, and this donor-based strategy enables an efficient on-the-fly preallocation. First, as donor files are preallocated and registered in the Maco structure, Éxitos quickly transfers blocks to the target log file and updates the Maco for both files. Second, Éxitos moves a contiguous range of blocks at a time to minimize the runtime cost. In light of our observations (see Section III), the allocation space shall be greater than the typical size of logging I/Os. We calculate a moving average of write requests online over a time window by a simple monitor module and empirically set the transferred space four times of the average logging I/O size. When the log file is closed, Éxitos transfers back blocks that are not used to the source donor file, in order to avoid space waste in the donor pool.

Éxitos sets the size of the transferred space from the logging I/O sizes that the monitor module measures, replenishes the donor pool in the background, and reclaims unused blocks when a log file is closed. As described above, Éxitos decides online how much space to transfer at a time, by setting the transferred space to four times the average logging I/O size that the monitor module measures over a time window, and Éxitos returns the blocks that the log file has not used to the source donor file at the moment the log file is closed. Once the free space of the donor pool drops below a threshold, Éxitos preallocates new donor files to replenish the donor pool off the critical path. If the donor pool cannot satisfy a request for space, or if a move-extent operation fails, Éxitos falls back to Ext4’s normal allocation path for the request. The log file remains correct, because the correctness of a logging write never depends on the donor pool, and the request loses only the acceleration that Éxitos would otherwise provide. Since Éxitos issues move-extent operations ahead of time through `io_uring`, a write request may arrive before the move-extent operation that prepares space for the write request has completed; in such a case Éxitos lets the write request wait until the move-extent operation completes, so the write request never observes a partially updated mapping.

D. Optimizations and Discussions

Optimizations. We optimize Éxitos with two tactics. First, we leverage the `io_uring` [16, 18] to asynchronously transfer extents for non-preallocated log files. The `io_uring` provides user-space asynchronous I/O interfaces, while the eBPF used by Éxitos sits at the boundary between kernel and user spaces. As a result, while dealing with an ongoing write at runtime, Éxitos simultaneously executes the *move-extent* through `io_uring` to prepare for future writes. In brief, it parallelizes handling the current logging write with demanding space for incoming ones. This overlapped execution reduces the cost of invoking *move-extent* on the critical path. Second, for high scalability we partition the Maco and donor pool, and distribute them to be per-core structures. In other words, we make them decentralized to reduce contention issues.

Crash consistency. Éxitos changes where a logging write is delivered, but changes neither the content of the write nor the moment at which the write is declared persistent. Éxitos writes the same bytes to the same device blocks, because the block numbers that Éxitos uses come from Ext4’s own extent tree. For ordering, Éxitos synchronously waits for the return signals of write and `fdatsync` before returning to the database, so the order on which a database relies, i.e., completing the logging write ahead of the in-place update of database records, stays unchanged. For persistence, an SSD with PLP quickly completes a flush, and if an SSD does not have PLP, Éxitos explicitly issues a flush command, such as the `nvme-flush` of NVMe protocol, to drain the SSD’s internal cache, so that the shortcut I/O path provides the same persistence semantics as the original `fdatsync` [24]. For a partial write, Éxitos returns the same error to the database as the conventional path returns when the device reports a write as failed or incomplete. Databases already protect log records with checksums and, at recovery, recognize and discard an incomplete record at the tail of the log file, exactly as databases do after a crash on the conventional path. For metadata after a crash, the Maco structure holds no state that Ext4 cannot derive on its own: Éxitos builds the Maco structure from the extent tree when opening the log file, and for a growing log file the only operation that changes the Maco structure is the *move-extent* that Éxitos itself issues. *Move-extent* is an operation of Ext4, and Ext4 journals every *move-extent*. Ext4’s on-disk metadata is therefore authoritative after a crash, and Éxitos rebuilds the Maco structure when the log file is opened next time, so a crash leaves behind no mapping that only Éxitos knows. Éxitos writes data into transferred space only after the corresponding *move-extent* has completed, so the case never arises that data has already been written into blocks while Ext4 still regards the blocks as belonging to the donor file. If a *move-extent* operation fails, or if the donor pool cannot satisfy a request for space, Éxitos falls back to Ext4’s normal allocation path.

Stale mappings. The per-file mappings that Éxitos caches in the Maco structure do not become stale during the period in which Éxitos uses the mappings. For a preallocated log file, writing data into already allocated space triggers no block

allocation, and a database neither truncates nor reallocates a log file that the database is appending data to, so the mappings of a preallocated log file do not change while the log file stays open. For a growing log file, the only operation that changes the mappings is the *move-extent* that Éxitos itself issues, and Éxitos updates the Maco structure in the same step. Éxitos builds the Maco structure when opening the log file and discards the Maco structure when closing the log file, so the Maco structure exists exactly during the period in which the log file stays open. An operation that happens while the log file is not open, for example Ext4’s online defragmentation, therefore leaves no outdated entry in the Maco structure. In the strict mode, moreover, Éxitos verifies inode attributes through `ioctl` interfaces at every operation, and such verification also observes a change of file metadata.

Error reporting. The `ioctl` interfaces return the completion status of the device to Éxitos, and Éxitos maps the completion status onto the same `errno` that the conventional path returns on write or `fdatsync`. A database therefore sees the same errors as the conventional path returns, and the error handling logic that the database already has needs no change.

Permission checks. Linux has two models for permission checks, i.e., the Discretionary Access Control (DAC) and Mandatory Access Control (MAC). DAC checks permission at opening a file. MAC does so at each file operation. Databases hardly change the access permission of log files, e.g., from read/write to read-only. Though, we make configurable dual modes for Éxitos. Its *fast mode* aligns with DAC’s one-time validation for performance. The *strict mode* follows MAC compliance for enhanced protection through inode attribute verification via `ioctl` interfaces at every operation.

Compatibility. Éxitos is functional with both SATA and NVMe SSDs. This is a contrast to prior designs like BypassD working for NVMe SSDs only. For SATA SSD, Éxitos uses the SCSI abstraction layer rather than sending ATA commands directly, because many Linux distributions restrict direct ATA transmission [25].

Portability. Some parts of Éxitos are general, while other parts depend on a specific file system, operating system, or device. Caching the per-file offset-to-block mappings and consulting the cached mappings at runtime only require that the mappings of a file stay stable while the file is open, and any file system that allocates blocks and updates data in place, including Ext4 and XFS, meets the requirement. The donor-based online space allocation additionally requires that the file system provide an interface that exchanges a range of space between two files without copying data. Ext4 provides *move-extent* [17], and XFS provides a range-exchange `ioctl` that serves the same purpose [26]. On file systems that write data out of place instead of updating data in place, such as Btrfs and F2FS, every overwrite changes the mappings of a file, so the requirement of stable mappings does not hold and Éxitos would need another way to obtain block numbers. Intercepting syscalls with eBPF and delivering data through `ioctl` interfaces are both specific to Linux. The flush command is specific to

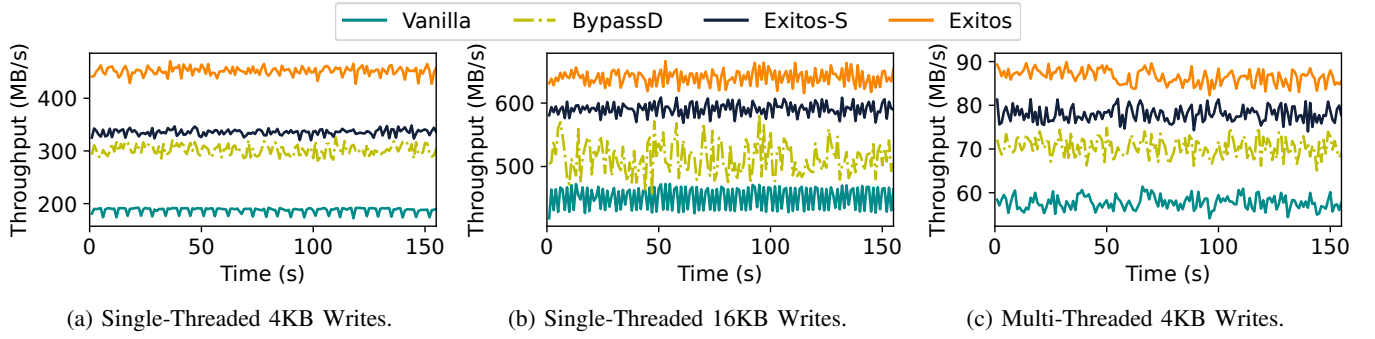


Fig. 6: Real-time throughputs with Fio tests that simulate three different logging write patterns.

[the device, and Éxitos already issues the commands of both NVMe and SCSI.](#)

V. EVALUATION

We evaluate Éxitos with real-world databases and prevalent benchmarks (Section V-A). We particularly aim to answer the following four questions.

- Does Éxitos indeed enable a fast shortcut path that outperforms conventional path and SOTA design? (Section V-B)
- Does Éxitos manage to boost performance for a database with preallocated log files? (Section V-C)
- Is Éxitos’s strategy of reusing Ext4’s *move-extent* feature efficient to resemble online preallocations? (Section V-D)
- Does enhanced Éxitos improve performance for a database using a non-preallocated log file? (Section V-E)

A. Experimental Setup

Configurations. We conduct all experiments on an HP Z2 G4 workstation equipped with an Intel Core™ i9-9900K CPU (16 cores) and 64GB DRAM. For storage, we set up Éxitos on low-latency Samsung PM1725a with the PLP hardware support. The OS is Ubuntu 22.04.1 and the kernel version is 6.6.5. The compiler is GCC/G++ 11.4.0.

Testbeds. We take OceanBase (version 4.3.3) and MySQL (version 8.4) to evaluate Éxitos. The reason why they are used is threefold. First, both of them are widely deployed. Second, they are differently built on LSM-tree and B-tree, respectively. Third, OceanBase mainly utilizes a preallocated log file, while MySQL has one preallocated log (the redo log of InnoDB [2]) and the other non-preallocated binlog, so with both databases we can comprehensively assess all aspects of Éxitos. Additionally, we do not change their source codes, but edit a configuration file in them to support Éxitos identifying and intercepting logging writes.

Baselines. We evaluate Éxitos in two modes, i.e., Éxitos for fast mode (DAC) and Éxitos-S for strict permission checks (MAC). Regarding the concrete difficulty of reprogramming a complex database with SPDK or NVMeDirect (see Table I), we mainly compare Éxitos to the stock storage stack (denoted as ‘Vanilla’) and BypassD [14]. The latter provides source codes with specialized IOMMU by emulation [27].

Benchmarks. We mainly use two benchmarks for evaluation. The micro-benchmark is Fio. The macro-benchmark is SysBench [28] that has multiple workloads reflecting production scenarios. We run SysBench on both databases.

B. Micro-benchmarking (Fio)

We simulate three typical patterns of database logging writes using Fio. Let us first study how Éxitos handles logging I/Os by which databases frequently flush small log records. We configure a 1GB preallocated file. Each write is 4KB followed by `fdatsync`. Figure 6a shows that Éxitos exhibits the highest real-time throughput at the top, achieving 2.4× throughput compared to Vanilla. Éxitos hence manages to exempt the software tax imposed on logging writes. Its effectiveness is jointly achieved by all aspects of it. First, leveraging the preallocation for a file results in a stable structural layout with deterministic block numbers. Second, using the in-memory Maco structure minimizes the tedious cost of searching for a target block in the file’s extent tree. Third, the eBPF sitting between kernel and user spaces makes a bridge to pass each I/O request through the `ioctl` interfaces, which then allow delivering data to the device with specific block numbers. They help to avoid the traversal of software layers including VFS, Ext4, bio layer, and I/O scheduler, thereby saving a lot of time.

In Figure 6a, a comparison between the top curve and the middle fluctuating one tells that Éxitos surpasses BypassD, by 49.2% higher bandwidth on average for the entire writing task. BypassD depends on a specialized IOMMU for acceleration. Extending IOMMU to map in-file offsets to block numbers risks interfering with IOMMU’s main function. This may not only cause compatibility issues, but also degrade BypassD’s performance and portability. In contrast, Éxitos is a fully all-in-software design. It is readily deployable on existing systems and also yields higher throughput than BypassD.

In the second test, we engage a thread in writing data to a 1GB preallocated file, but the I/O size per write is 16KB. This is based on our measurement that OLTP write transactions empirically carry 16KB data on average. Figure 6b confirms that Éxitos outperforms Vanilla by 42.3% higher throughput. Also, the real-time throughput of Éxitos is 24.8% higher than that of BypassD on average, indicating that Éxitos is still more effective even in case of larger I/Os.

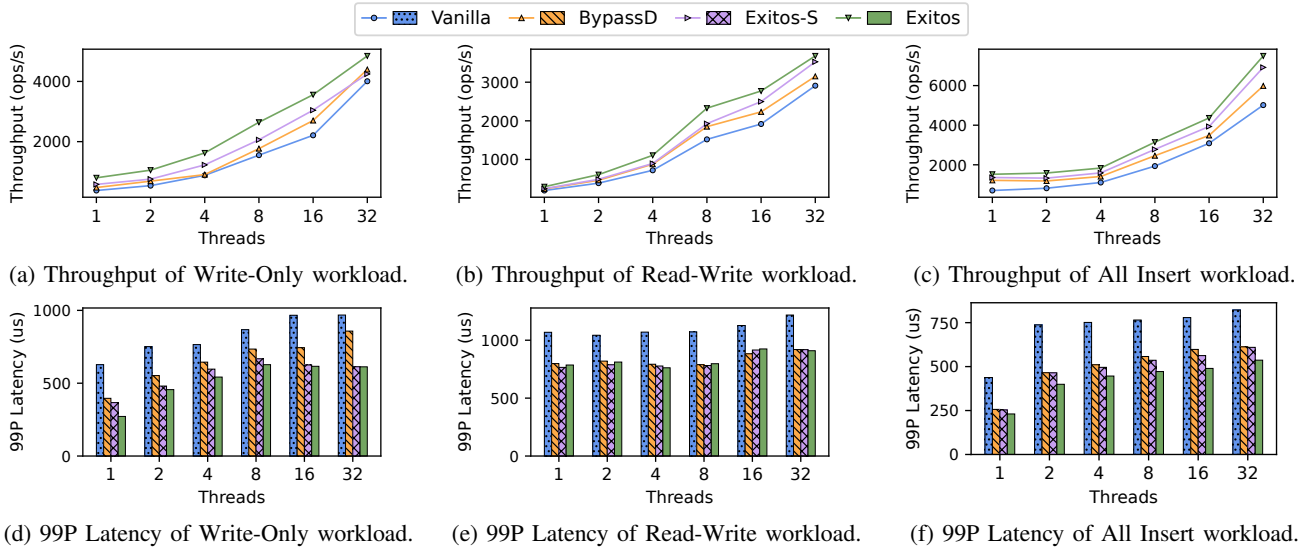


Fig. 7: Throughput and 99P tail latency of Éxitos on OceanBase with varying thread counts under SysBench workloads.

Éxitos is optimized for scalability with per-core structures. To assess its performance with multi-threads, we run 16 threads in parallel, each writing its own file, to match the number of CPU cores. As shown in Figure 6c, Éxitos produces 47.4% higher throughput than Vanilla. As Éxitos builds parallel shortcut I/O paths for multi-threads and utilizes the distributed Maco structure, it manages to carry on concurrent write I/Os with low contention and yield consistently high performance.

Lastly, let us compare Éxitos with Éxitos-S. The gaps are 25.8%, 7.7%, and 34.1% between them with three test cases, reflecting the moderate cost of stricter permission checks for Éxitos-S. That said, this cost is avoidable in practice, because real-world databases are unlikely to alter the write/read permissions of log files at runtime.

C. Macro-benchmarking (OceanBase)

Next we employ Éxitos to substitute the path of OceanBase’s logging writes. We run SysBench on OceanBase atop four configurations, i.e., the Vanilla, BypassD, Éxitos-S, and Éxitos. We take three representative OLTP workloads from SysBench for evaluation, i.e., OLTP-read_write (mixed read/write transactions), OLTP-all_insert (all insert transactions), and OLTP-write_only (insert and update transactions). We vary the client threads as 1, 2, 4, 8, 16, and 32. For each client count, we use a 10-million-record table and execute each workload for 30 minutes. Figure 7 shows the throughput in operations per second (OPS) and 99P (the 99th percentile) tail latency. Regarding these results, we derive five observations.

First, for write-intensive workloads with varying clients, Éxitos consistently surpasses Vanilla by offering higher throughput and lower tail latency. For the OLTP-write_only workload with a single client, Éxitos achieves a throughput of 2.1× compared to Vanilla, while the latter’s 99P latency is up to 2.3× that of Éxitos. Similar performance trends are also observed with other two workloads. Éxitos minimizes the software overhead, as it skips over multiple software layers by routing

logging I/Os through its own fast path. Second, with 32 clients, the throughput and tail latency gaps reach 21.4% and 62.3%, respectively. Éxitos is efficient and optimized for scalability. OceanBase must handle contention among multiple threads, which leads to inevitable penalty in execution and offsets the effect of Éxitos. However, Éxitos is still performant. Third, for OLTP-read_write composed of mixed write and read transactions, Éxitos remains effective. It optimizes logging writes, while databases do not query a log file for read operations. In fact, Éxitos places emphasis on logging writes and incurs no change to the original read path of OceanBase or other databases. OceanBase’s user-space cache and the OS’s page cache help to retain high hit rate and read throughput. Fourth, Éxitos outperforms BypassD by 31.1% on average, which aligns with micro-benchmarking results. Finally, OceanBase never alters access permissions or resizes log files. As shown in Figure 7, stricter checks for Éxitos-S still introduce marginal overhead, e.g., 18.5% on OLTP-read_write at 32 clients. Hence, users can consider either mode as needed, while Éxitos-S provides stronger protection at low cost.

D. The Effect of Online Preallocation

We have presented a test of on-the-fly space allocation in Section III (see Figure 2). Before testing with a real-world growing log file, we evaluate the effect of Éxitos’s preallocation-like strategy that makes use of the *move-extent* feature of Ext4 as well as the *io_uring* technique. We repeat the same test onto Éxitos with a continuous series of appending writes accumulated to overall 10GB, with 4KB, 8KB, 16KB, 32KB, 64KB, or 128KB per write. We configure Éxitos to asynchronously preallocate 64KB, 256KB, or 1MB from donor files. Figure 8 shows the curves for three preallocation sizes. We also include the results of conventional non-preallocation way (the curve with dots). Compared to it, Éxitos increases the throughput, for example, in terms of the typical 4KB I/O size, by 3.3×, 3.6×, and 3.9× with three increasing allocation

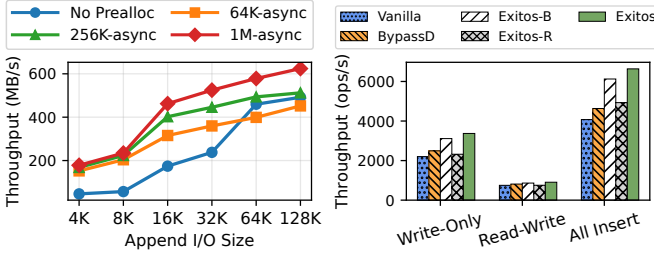


Fig. 8: Impact of bypassing different log types.

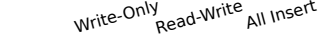


Fig. 9: MySQL Throughput with SysBench workloads.

spaces, respectively. ~~They are all higher than corresponding improvements obtained by preallocation on Section III~~ has tested another way to reserve space, i.e., calling fallocate on the critical path to reserve a batch of blocks prior to each appending write (see Figure 2), which serves as a batched preallocation baseline. Calling fallocate in this way raises the throughput of 4KB writes by 2.6×, 3.2×, and 3.4× over the conventional non-preallocation way with the preallocation sizes of 64KB, 256KB, and 1MB, whereas Éxitos reaches 3.3×, 3.6×, and 3.9× with the same three preallocation sizes. Both tests reserve the same amount of space and issue writes of the same size, so the difference between the two groups of numbers does not come from performing fewer allocations. The difference comes from moving the allocation off the critical path and from the shortcut I/O path that Éxitos builds. These results justify that, by employing donor files as a supplier of spaces and avoids on-demand allocations, Éxitos gains promising efficiency and performance.

E. Macro-benchmarking (MySQL)

To comprehensively verify Éxitos’s capability, we take MySQL for more tests. We set up MySQL 8.4 with default parameters. It has two main logs. One is the binlog that uses appending writes for replication and recovery, while the InnoDB engine has a redo log that is preallocated and reused as a circular buffer [2]. This dual-logging architecture exhibits an ideal scenario for evaluating Éxitos on both preallocated and growing log files.

Before testing with Éxitos, let us figure out the impacts of two logs on the performance of MySQL. We forcefully disable either log or both and execute OLTP workloads of SysBench. In the absence of both logs, the performance boosts by 109.0% with the OLTP-write_only workload. With either binlog or redo log disabled, the performance rises up by 60.8% and 12.8%, respectively. Consequently, the aggregated cost of using both logs is significant, while the binlog brings more performance overhead than the redo log.

We next run three SysBench workloads as used in Section V-C. Since there are two logs in MySQL, we make three variants, i.e., Éxitos handling writes to both logs, Éxitos-B handling binlog only, and Éxitos-R handling redo log only. All of them apply the default DAC for permission checks. In particular, Éxitos and Éxitos-B frequently call the *move-extent* and *io_uring* to extend the binlog file in a timely and

asynchronous fashion, resembling a preallocation effect on the fly. Like what we have done with OceanBase, each test case runs for 30 minutes on a table composed of 10 million records.

Figure 9 depicts the throughput comparison across five configurations. With the OLTP-write_only workload, Éxitos achieves 53.3% higher throughput than Vanilla, demonstrating substantial performance gains by reshaping the path for both logs. Éxitos-B that optimizes only binlog writes, delivers 41.5% improvement, while Éxitos-R shows modest 5.5% gains by targeting the redo log only. This aligns with what we have obtained by disabling both logs or either one.

For the OLTP-read_write workload with mixed write and read transactions, Éxitos achieves 21.0% higher throughput. Read operations do not benefit from Éxitos’s write-optimized shortcut path, and the workload’s mixed nature reduces the relative impact of logging optimizations. As to the OLTP-all_insert workload, Éxitos yields the maximum potential, achieving 63.1% (i.e., 1.6×) throughput improvement than Vanilla. This workload heavily stresses both logs for MySQL, since each insert triggers file writes to binlog and redo log. Additionally, with all three workloads, Éxitos again outperforms BypassD, further confirming Éxitos’s superiority.

To summarize, Éxitos reaps further efficiency and applicability with its preallocation-like strategy for non-preallocated log files. Overall, the consistent performance gains with both OceanBase and MySQL justify that Éxitos’s shortcut path effectively accelerates database logging writes.

VI. CONCLUSION

In this paper, we demonstrate that the heavy software tax charged on database logging writes is amplified with fast storage devices and can be relieved in an all-in-software way. We accordingly develop Éxitos. Éxitos utilizes eBPF to transparently redirect and transform file operations into actual I/Os with the storage device, such that it manages to bypass the most software layers. By exploiting the *move-extent* feature of Ext4, Éxitos minimizes the software tax for both preallocated and non-preallocated log files. Experiments confirm that Éxitos largely improves the performance of representative databases.

ACKNOWLEDGMENT

This work was supported by National Key R&D Program of China No. 2022YFB4401700 and ShanghaiTech Startup Funding. Chundong Wang is the corresponding author.

REFERENCES

- [1] Z. Yang *et al.*, “OceanBase: a 707 million tpmC distributed relational database system,” *Proc. VLDB Endow.*, vol. 15, no. 12, p. 3385–3397, Aug. 2022.
- [2] Oracle Corporation, *MySQL*. Oracle Corporation, 2025. [Online]. Available: <https://www.mysql.com/>
- [3] W. Lee *et al.*, “WALDIO: Eliminating the filesystem journaling in resolving the journaling of journal anomaly,” in *2015 USENIX Annual Technical Conference (USENIX ATC 15)*. Santa Clara, CA: USENIX Association, Jul. 2015, pp. 235–247.
- [4] D. Kim, Y. Won, J. Cha, S. Yoon, J. Choi, and S. Kang, “Exploiting compression-induced internal fragmentation for power-off recovery in SSD,” *IEEE Trans. Comput.*, vol. 65, no. 6, p. 1720–1733, Jun. 2016.

- [5] Samsung, “Power loss protection (PLP): Protect your data against sudden power loss,” https://download.semiconductor.samsung.com/resources/others/Samsung_SSD_845DC_05_Power_loss_protection_PLP.pdf, 2014.
- [6] E. B. Nightingale *et al.*, “Rethink the sync,” in *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*. Seattle, WA: USENIX Association, Nov. 2006.
- [7] V. Chidambaram *et al.*, “Optimistic crash consistency,” in *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, ser. SOSP ’13. New York, NY, USA: ACM, 2013, p. 228–243.
- [8] H. Shirwadkar *et al.*, “FastCommit: resource-efficient, performant and cost-effective file system journaling,” in *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. Santa Clara, CA: USENIX Association, Jul. 2024, pp. 157–171.
- [9] D. Park and D. Shin, “iJournaling: Fine-Grained journaling for improving the latency of fsync system call,” in *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. Santa Clara, CA: USENIX Association, Jul. 2017, pp. 787–798.
- [10] J. Yeon *et al.*, “RFLUSH: Rethink the flush,” in *16th USENIX Conference on File and Storage Technologies (FAST 18)*. Oakland, CA: USENIX Association, Feb. 2018, pp. 201–210.
- [11] Z. Yang *et al.*, “SPDK: A development kit to build high performance storage applications,” in *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 2017, pp. 154–161.
- [12] H.-J. Kim *et al.*, “NVMeDirect: A user-space I/O framework for application-specific optimization on NVMe SSDs,” in *8th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 16)*. Denver, CO: USENIX Association, Jun. 2016.
- [13] A. M. Caulfield *et al.*, “Providing safe, user space access to fast, solid state disks,” in *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. New York, NY, USA: ACM, 2012, p. 387–400.
- [14] S. Yadalam *et al.*, “BypassD: Enabling fast userspace access to shared SSDs,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. New York, NY, USA: ACM, 2024, p. 35–51.
- [15] eBPF, “eBPF,” <https://ebpf.io/>, Aug. 2025.
- [16] Linux manual page, “ioctl: System calls manual,” <https://www.man7.org/linux/man-pages/man2/ioctl.2.html>, June 2025.
- [17] Linux Kernel Documentation, “Ext4_ioc_move_ext - online defragmentation for ext4,” 2026, <https://www.kernel.org/doc/html/latest/filesystems/ext4.html>.
- [18] J. Axboe, “Efficient IO with io_uring,” 2019, https://kernel.dk/io_uring.pdf.
- [19] Y. Zhong *et al.*, “XRP: In-Kernel storage functions with eBPF,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 375–393.
- [20] J. Axboe, “fio - flexible I/O tester rev. 3.36,” https://fio.readthedocs.io/en/latest/fio_doc.html, August 2024.
- [21] Intel Corporation, “Get started with Intel VTune profiler,” <https://www.intel.com/content/www/us/en/docs/vtune-profiler/>, Mar 2026.
- [22] C. Arges, “Tracing linux: Fast, compatible, complete,” Mar. 2020, <https://www.conflura.com/post/tracing-linux-fast-compatible-complete>.
- [23] Y. Zheng *et al.*, “Extending applications safely and efficiently,” in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. Boston, MA: USENIX Association, Jul. 2025, pp. 557–574.
- [24] The kernel development community, “Explicit volatile write back cache control,” https://www.kernel.org/doc/html/latest/block/writeback_cache_control.html, Jan 2026.
- [25] Linux Kernel Documentation, “libata developer’s guide,” 2025, <https://www.kernel.org/doc/html/latest/driver-api/libata.html>.
- [26] L. K. Documentation, “ioctl_xfs_exchange_range(2): Exchange the contents of parts of two files,” https://man7.org/linux/man-pages/man2/ioctl_xfs_exchange_range.2.html, Aug. 2025.
- [27] The SCAIL Reseach Group, “Bypassd: Enabling fast userspace access to shared SSDs,” <https://github.com/multifacet/BypassD>, 2023.
- [28] P. Zaitsev, “Sysbench,” <https://github.com/akopytov/sysbench>, July 2024.